

```

package REPASO7;

// Ejercicio 1
public class Caja<T> {
    private T contenido;
    public Caja(T contenido) { this.contenido = contenido; }
    public T getContenido() { return contenido; }
    public void setContenido(T contenido) { this.contenido =
contenido; }
    public void mostrarContenido() { System.out.println(contenido);
}
}

// Ejercicio 2
public class Par<K, V> {
    private K clave;
    private V valor;
    public Par(K clave, V valor) { this.clave = clave; this.valor =
valor; }
    public K getClave() { return clave; }
    public V getValor() { return valor; }
    public String toString() { return clave + " -> " + valor; }
}

// Ejercicio 3
public static <T> void mostrarArray(T[] array) {
    for (T e : array) System.out.println(e);
}

// Ejercicio 4
public static <T> T obtenerPrimero(List<T> lista) {
    if (lista.isEmpty()) return null;
    return lista.get(0);
}

// Ejercicio 5
public static <T> int contarCoincidencias(List<T> lista, T
elemento) {
    int count = 0;
    for (T e : lista) if (e.equals(elemento)) count++;
    return count;
}

// Ejercicio 6
public static <T extends Comparable<T>> T mayor(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}

```

```

// Ejercicio 7
class Repositorio<T> {
    private List<T> lista = new ArrayList<>();
    public void añadir(T e) { lista.add(e); }
    public void eliminar(T e) { lista.remove(e); }
    public List<T> obtenerTodos() { return lista; }
    public T obtenerPorPosicion(int i) { return lista.get(i); }
    public int contar() { return lista.size(); }
}

// Ejercicio 8
public static <T> List<T> filtrar(List<T> lista, Predicate<T>
condicion) {
    return
lista.stream().filter(condicion).collect(Collectors.toList());
}

// Ejercicio 9
public static <T, R> List<R> transformar(List<T> lista,
Function<T, R> funcion) {
    return
lista.stream().map(funcion).collect(Collectors.toList());
}

// Ejercicio 10
public static <T> List<T> ordenarCopia(List<T> lista,
Comparator<T> comparador) {
    return
lista.stream().sorted(comparador).collect(Collectors.toList());
}

// PARTE 2 - REFERENCIAS DE MÉTODO

// Ejercicio 11
Function<String, String> f11 = String::toUpperCase;

// Ejercicio 12
Function<String, String> f12 = String::trim;

// Ejercicio 13
Consumer<String> c13 = System.out::println;

// Ejercicio 14
Function<Alumno, String> f14 = Alumno::getNombre;

// Ejercicio 15
Predicate<Alumno> p15 = Alumno::estaAprobado;

```

```

// Ejercicio 16
Consumer<Alumno> c16 = Alumno::mostrarFicha;

// Ejercicio 17
Function<String, String> f17 = UtilidadesTexto::limpiar;

// Ejercicio 18
Predicate<String> p18 = UtilidadesTexto::esLargo;

// Ejercicio 19
Impresora impresora = new Impresora();
Consumer<String> c19 = impresora::imprimir;

// Ejercicio 20
Supplier<ArrayList<String>> s20 = ArrayList::new;

// Ejercicio 21
// No se puede convertir directamente porque el constructor recibe 3
// parámetros
// y la lambda solo recibe 1, así que no hay referencia de método
// válida.
Function<String, Alumno> f21 = nombre -> new Alumno(nombre, 18,
5.0);

// Ejercicio 22
Comparator<Alumno> c22 = Comparator.comparing(Alumno::getNombre);

// Ejercicio 23
BiPredicate<String, String> p23 = String::equalsIgnoreCase;

// Ejercicio 24
Function<String, Integer> f24 = Integer::parseInt;

// Ejercicio 25
BiFunction<String, String, Integer> f25 =
String::compareToIgnoreCase;

// PARTE 3 - STREAMS

// Ejercicio 26
List<String> ej26 = productos.stream()
    .filter(p -> p.getPrecio() > 50)
    .map(ProductoTienda::getNombre)
    .sorted()
    .collect(Collectors.toList());

// Ejercicio 27

```

```

List<String> ej27 = productos.stream()
    .filter(p -> p.getCategoria().equals("Electrónica") &&
p.getStock() > 5)
    .map(ProductoTienda::getNombre)
    .collect(Collectors.toList());

// Ejercicio 28
double ej28 = productos.stream()
    .mapToDouble(ProductoTienda::getPrecio)
    .average().orElse(0);

// Ejercicio 29
Map<String, List<ProductoTienda>> ej29 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria));

// Ejercicio 30
Map<String, Double> ej30 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
Collectors.averagingDouble(ProductoTienda::getPrecio)));

// Ejercicio 31
Map<String, Integer> ej31 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.summingInt(ProductoTienda::getStock)));

// Ejercicio 32
List<String> ej32 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria))
        .entrySet().stream()
        .filter(e -> e.getValue().size() >= 3)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());

// Ejercicio 33
List<String> ej33 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria))
        .entrySet().stream()
        .filter(e -> e.getValue().size() >= 3)
        .flatMap(e -> e.getValue().stream())
        .map(ProductoTienda::getNombre)
        .sorted()
        .collect(Collectors.toList());

```

```
// Ejercicio 34
Map<String, Optional<ProductoTienda>> ej34 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria,

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPre
cio))));
```

```
// Ejercicio 35
Map<String, String> ej35 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria,
    Collectors.collectingAndThen(

Collectors.minBy(Comparator.comparingDouble(ProductoTienda::getPre
cio)),
        o ->
o.map(ProductoTienda::getNombre).orElse(""))));
```

```
// Ejercicio 36
List<ProductoTienda> ej36 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .collect(Collectors.toList());
```

```
// Ejercicio 37
List<String> ej37 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .map(ProductoTienda::getNombre)
    .distinct().sorted()
    .collect(Collectors.toList());
```

```
// Ejercicio 38
double ej38 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .mapToDouble(p -> p.getPrecio() * p.getStock())
    .sum();
```

```
// Ejercicio 39
Map<String, Double> ej39 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingDouble(p ->
p.getProductos().stream()
            .mapToDouble(pr -> pr.getPrecio() *
pr.getStock()).sum())));
```

```
// Ejercicio 40
List<String> ej40 = ej39.entrySet().stream()
```

```

        .filter(e -> e.getValue() > 500)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());

// Ejercicio 41
Map<String, Long> ej41 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingLong(p ->
p.getProductos().size())));

// Ejercicio 42
List<String> ej42 = pedidos.stream()
    .filter(p -> p.getProductos().stream()
        .anyMatch(pr ->
pr.getCategoria().equals("Electrónica")))
    .map(Pedido::getCliente)
    .distinct()
    .collect(Collectors.toList());

// Ejercicio 43
List<String> ej43 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente))
    .entrySet().stream()
    .filter(e -> e.getValue().size() > 1)
    .flatMap(e -> e.getValue().stream())
    .flatMap(p -> p.getProductos().stream())
    .map(ProductoTienda::getNombre)
    .distinct().sorted()
    .collect(Collectors.toList());

// Ejercicio 44
List<ProductoTienda> ej44 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .filter(p -> p.getStock() > 1)

    .sorted(Comparator.comparingDouble(ProductoTienda::getPrecio).reverse())
    .collect(Collectors.toList());

// Ejercicio 45
Map<String, Optional<ProductoTienda>> ej45 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.flatMapping(p ->
p.getProductos().stream(),

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPrecio)))));

```

// Ejercicio 46

```
List<Double> ej46 = alumnos.stream()
    .flatMap(a -> a.getNotas().stream())
    .collect(Collectors.toList());
```

// Ejercicio 47

```
double ej47 = alumnos.stream()
    .flatMapToDouble(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue))
    .average().orElse(0);
```

// Ejercicio 48

```
Map<String, Double> ej48 = alumnos.stream()
    .collect(Collectors.toMap(AlumnoCurso::getNombre,
        a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0)));
```

// Ejercicio 49

```
Map<String, Double> ej49 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.averagingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0))));
```

// Ejercicio 50

```
List<String> ej50 = alumnos.stream()
    .filter(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0) >= 7)
    .map(AlumnoCurso::getNombre)
    .collect(Collectors.toList());
```

// Ejercicio 51

```
String ej51 = alumnos.stream()
    .max(Comparator.comparingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0)))
    .map(AlumnoCurso::getNombre).orElse("");
```

// Ejercicio 52

```
Map<String, List<String>> ej52 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.mapping(AlumnoCurso::getNombre,
Collectors.toList())));
```

// Ejercicio 53

```
List<String> ej53 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso))
    .entrySet().stream()
    .filter(e -> e.getValue().stream().allMatch(a ->
        a.getNotas().stream().mapToDouble(Double::doubleValue).average().orElse(0) >= 5))
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 54

```
List<String> ej54 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso))
    .entrySet().stream()
    .filter(e -> e.getValue().stream().anyMatch(a ->
        a.getNotas().stream().anyMatch(n -> n < 5)))
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 55

```
List<Empleado> ej55 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.toList());
```

// Ejercicio 56

```
List<String> ej56 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .filter(e -> e.getSalario() > 2000)

    .sorted(Comparator.comparingDouble(Empleado::getSalario).reversed())
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

// Ejercicio 57

```
Map<String, List<Empleado>> ej57 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto));
```

// Ejercicio 58

```
Map<String, Double> ej58 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
        d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average().orElse(0)));
```

// Ejercicio 59


```

String ej59 = ej58.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey).orElse("");

// Ejercicio 60
List<Empleado> ej60 = departamentos.stream()
    .filter(d -> d.getEmpleados().size() >= 3)
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.toList());

// Ejercicio 61
List<String> ej61 = departamentos.stream()
    .filter(d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
e().orElse(0) > 2000)
    .flatMap(d -> d.getEmpleados().stream())
    .map(Empleado::getNombre)
    .collect(Collectors.toList());

// Ejercicio 62
Map<String, Long> ej62 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d -> (long) d.getEmpleados().size()));

// Ejercicio 63
Map<String, Double> ej63 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).max().
orElse(0)));

// Ejercicio 64
List<String> ej64 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto,
Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 65
Map<String, List<Empleado>> ej65 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 2)

```

```

        .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue));

// Ejercicio 66
List<String> ej66 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria))
        .entrySet().stream()
        .filter(e ->
e.getValue().stream().mapToDouble(ProductoTienda::getPrecio).average()
        .orElse(0) > 50)
        .flatMap(e -> e.getValue().stream())
        .map(ProductoTienda::getNombre)
        .sorted()
        .collect(Collectors.toList());

// Ejercicio 67
List<String> ej67 = pedidos.stream()
        .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingDouble(p ->
p.getProductos().stream()
        .mapToDouble(pr -> pr.getPrecio() *
pr.getStock()).sum()))
        .entrySet().stream()
        .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());

// Ejercicio 68
List<String> ej68 = pedidos.stream()
        .flatMap(p -> p.getProductos().stream())

        .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.summingInt(ProductoTienda::getStock)))
        .entrySet().stream()
        .filter(e -> e.getValue() > 5)
        .map(Map.Entry::getKey)
        .collect(Collectors.toList());

// Ejercicio 69
Map<String, List<String>> ej69 = alumnos.stream()
        .filter(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
        rElse(0) >= 5)
        .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.mapping(AlumnoCurso::getNombre,
Collectors.toList())));

```

```
// Ejercicio 70
List<String> ej70 = departamentos.stream()
    .filter(d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
e().orElse(0) > 1800)
    .flatMap(d -> {
        double media =
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
e().orElse(0);
        return d.getEmpleados().stream()
            .filter(e -> e.getSalario() > media &&
e.getAntiguedad() >= 3);
    })
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

```
// Ejercicio 71
Map<String, List<String>> ej71 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.flatMapping(p ->
p.getProductos().stream().map(ProductoTienda::getCategoria).distinct(),
            Collectors.toList())));
```

```
// Ejercicio 72
Map<String, String> ej72 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPrecio)),
            o ->
o.map(ProductoTienda::getNombre).orElse(""))));
```

```
// Ejercicio 73
List<String> ej73 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.averagingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().orElse(0))))
    .entrySet().stream()
    .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

```
// Ejercicio 74
Map<String, List<String>> ej74 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().map(Empleado::getNombre).sorted().collect(Collectors.toList())));
```

```
// Ejercicio 75
String ej75 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingInt(p ->
p.getProductos().stream().mapToInt(ProductoTienda::getStock).sum()
)))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey).orElse("");
package REPAS07;
```

```
// Ejercicio 1
public class Caja<T> {
    private T contenido;
    public Caja(T contenido) { this.contenido = contenido; }
    public T getContenido() { return contenido; }
    public void setContenido(T contenido) { this.contenido =
contenido; }
    public void mostrarContenido() { System.out.println(contenido);
}
}
```

```
// Ejercicio 2
public class Par<K, V> {
    private K clave;
    private V valor;
    public Par(K clave, V valor) { this.clave = clave; this.valor =
valor; }
    public K getClave() { return clave; }
    public V getValor() { return valor; }
    public String toString() { return clave + " -> " + valor; }
}
```

```
// Ejercicio 3
public static <T> void mostrarArray(T[] array) {
    for (T e : array) System.out.println(e);
}
```

```
// Ejercicio 4
```

```

public static <T> T obtenerPrimero(List<T> lista) {
    if (lista.isEmpty()) return null;
    return lista.get(0);
}

// Ejercicio 5
public static <T> int contarCoincidencias(List<T> lista, T
elemento) {
    int count = 0;
    for (T e : lista) if (e.equals(elemento)) count++;
    return count;
}

// Ejercicio 6
public static <T extends Comparable<T>> T mayor(T a, T b) {
    return a.compareTo(b) >= 0 ? a : b;
}

// Ejercicio 7
class Repositorio<T> {
    private List<T> lista = new ArrayList<>();
    public void añadir(T e) { lista.add(e); }
    public void eliminar(T e) { lista.remove(e); }
    public List<T> obtenerTodos() { return lista; }
    public T obtenerPorPosicion(int i) { return lista.get(i); }
    public int contar() { return lista.size(); }
}

// Ejercicio 8
public static <T> List<T> filtrar(List<T> lista, Predicate<T>
condicion) {
    return
lista.stream().filter(condicion).collect(Collectors.toList());
}

// Ejercicio 9
public static <T, R> List<R> transformar(List<T> lista,
Function<T, R> funcion) {
    return
lista.stream().map(funcion).collect(Collectors.toList());
}

// Ejercicio 10
public static <T> List<T> ordenarCopia(List<T> lista,
Comparator<T> comparador) {
    return
lista.stream().sorted(comparador).collect(Collectors.toList());
}

```

```
// PARTE 2 - REFERENCIAS DE MÉTODO

// Ejercicio 11
Function<String, String> f11 = String::toUpperCase;

// Ejercicio 12
Function<String, String> f12 = String::trim;

// Ejercicio 13
Consumer<String> c13 = System.out::println;

// Ejercicio 14
Function<Alumno, String> f14 = Alumno::getNombre;

// Ejercicio 15
Predicate<Alumno> p15 = Alumno::estaAprobado;

// Ejercicio 16
Consumer<Alumno> c16 = Alumno::mostrarFicha;

// Ejercicio 17
Function<String, String> f17 = UtilidadesTexto::limpiar;

// Ejercicio 18
Predicate<String> p18 = UtilidadesTexto::esLargo;

// Ejercicio 19
Impresora impresora = new Impresora();
Consumer<String> c19 = impresora::imprimir;

// Ejercicio 20
Supplier<ArrayList<String>> s20 = ArrayList::new;

// Ejercicio 21
// No se puede convertir directamente porque el constructor recibe 3
// parámetros
// y la lambda solo recibe 1, así que no hay referencia de método
// válida.
Function<String, Alumno> f21 = nombre -> new Alumno(nombre, 18,
5.0);

// Ejercicio 22
Comparator<Alumno> c22 = Comparator.comparing(Alumno::getNombre);

// Ejercicio 23
BiPredicate<String, String> p23 = String::equalsIgnoreCase;
```

```

// Ejercicio 24
Function<String, Integer> f24 = Integer::parseInt;

// Ejercicio 25
BiFunction<String, String, Integer> f25 =
String::compareToIgnoreCase;

// PARTE 3 - STREAMS

// Ejercicio 26
List<String> ej26 = productos.stream()
    .filter(p -> p.getPrecio() > 50)
    .map(ProductoTienda::getNombre)
    .sorted()
    .collect(Collectors.toList());

// Ejercicio 27
List<String> ej27 = productos.stream()
    .filter(p -> p.getCategoria().equals("Electrónica") &&
p.getStock() > 5)
    .map(ProductoTienda::getNombre)
    .collect(Collectors.toList());

// Ejercicio 28
double ej28 = productos.stream()
    .mapToDouble(ProductoTienda::getPrecio)
    .average().orElse(0);

// Ejercicio 29
Map<String, List<ProductoTienda>> ej29 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria));

// Ejercicio 30
Map<String, Double> ej30 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
Collectors.averagingDouble(ProductoTienda::getPrecio)));

// Ejercicio 31
Map<String, Integer> ej31 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
Collectors.summingInt(ProductoTienda::getStock)));

// Ejercicio 32
List<String> ej32 = productos.stream()

```

```
.collect(Collectors.groupingBy(ProductoTienda::getCategoria))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 3)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 33

```
List<String> ej33 = productos.stream()
```

```
.collect(Collectors.groupingBy(ProductoTienda::getCategoria))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 3)
    .flatMap(e -> e.getValue().stream())
    .map(ProductoTienda::getNombre)
    .sorted()
    .collect(Collectors.toList());
```

// Ejercicio 34

```
Map<String, Optional<ProductoTienda>> ej34 = productos.stream()
```

```
.collect(Collectors.groupingBy(ProductoTienda::getCategoria,
```

```
Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPre
cio))));
```

// Ejercicio 35

```
Map<String, String> ej35 = productos.stream()
```

```
.collect(Collectors.groupingBy(ProductoTienda::getCategoria,
    Collectors.collectingAndThen(
```

```
Collectors.minBy(Comparator.comparingDouble(ProductoTienda::getPre
cio)),
```

```
o ->
```

```
o.map(ProductoTienda::getNombre).orElse(""));
```

// Ejercicio 36

```
List<ProductoTienda> ej36 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .collect(Collectors.toList());
```

// Ejercicio 37

```
List<String> ej37 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .map(ProductoTienda::getNombre)
    .distinct().sorted()
    .collect(Collectors.toList());
```


// Ejercicio 38

```
double ej38 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .mapToDouble(p -> p.getPrecio() * p.getStock())
    .sum();
```

// Ejercicio 39

```
Map<String, Double> ej39 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingDouble(p ->
            p.getProductos().stream()
                .mapToDouble(pr -> pr.getPrecio() *
                    pr.getStock()).sum())));
```

// Ejercicio 40

```
List<String> ej40 = ej39.entrySet().stream()
    .filter(e -> e.getValue() > 500)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 41

```
Map<String, Long> ej41 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingLong(p ->
            p.getProductos().size())));
```

// Ejercicio 42

```
List<String> ej42 = pedidos.stream()
    .filter(p -> p.getProductos().stream()
        .anyMatch(pr ->
            pr.getCategoria().equals("Electrónica")))
    .map(Pedido::getCliente)
    .distinct()
    .collect(Collectors.toList());
```

// Ejercicio 43

```
List<String> ej43 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente))
    .entrySet().stream()
    .filter(e -> e.getValue().size() > 1)
    .flatMap(e -> e.getValue().stream())
    .flatMap(p -> p.getProductos().stream())
    .map(ProductoTienda::getNombre)
    .distinct().sorted()
    .collect(Collectors.toList());
```

// Ejercicio 44

```

List<ProductoTienda> ej44 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .filter(p -> p.getStock() > 1)

.sorted(Comparator.comparingDouble(ProductoTienda::getPrecio).reverse())
    .collect(Collectors.toList());

// Ejercicio 45
Map<String, Optional<ProductoTienda>> ej45 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.flatMapping(p ->
p.getProductos().stream(),

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPrecio)))));

// Ejercicio 46
List<Double> ej46 = alumnos.stream()
    .flatMap(a -> a.getNotas().stream())
    .collect(Collectors.toList());

// Ejercicio 47
double ej47 = alumnos.stream()
    .flatMapToDouble(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue))
    .average().orElse(0);

// Ejercicio 48
Map<String, Double> ej48 = alumnos.stream()
    .collect(Collectors.toMap(AlumnoCurso::getNombre,
        a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().orElse(0)));

// Ejercicio 49
Map<String, Double> ej49 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.averagingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().orElse(0))));

// Ejercicio 50
List<String> ej50 = alumnos.stream()
    .filter(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().orElse(0) >= 7)

```

```

        .map(AlumnoCurso::getNombre)
        .collect(Collectors.toList());

// Ejercicio 51
String ej51 = alumnos.stream()
    .max(Comparator.comparingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0)))
    .map(AlumnoCurso::getNombre).orElse("");

// Ejercicio 52
Map<String, List<String>> ej52 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.mapping(AlumnoCurso::getNombre,
Collectors.toList())));

// Ejercicio 53
List<String> ej53 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso))
    .entrySet().stream()
    .filter(e -> e.getValue().stream().allMatch(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0) >= 5))
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 54
List<String> ej54 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso))
    .entrySet().stream()
    .filter(e -> e.getValue().stream().anyMatch(a ->
        a.getNotas().stream().anyMatch(n -> n < 5)))
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 55
List<Empleado> ej55 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.toList());

// Ejercicio 56
List<String> ej56 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .filter(e -> e.getSalario() > 2000)

```

```
.sorted(Comparator.comparingDouble(Empleado::getSalario).reversed(
))
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

```
// Ejercicio 57
```

```
Map<String, List<Empleado>> ej57 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto));
```

```
// Ejercicio 58
```

```
Map<String, Double> ej58 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
    .orElse(0))));
```

```
// Ejercicio 59
```

```
String ej59 = ej58.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey).orElse("");
```

```
// Ejercicio 60
```

```
List<Empleado> ej60 = departamentos.stream()
    .filter(d -> d.getEmpleados().size() >= 3)
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.toList());
```

```
// Ejercicio 61
```

```
List<String> ej61 = departamentos.stream()
    .filter(d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
    .orElse(0) > 2000)
    .flatMap(d -> d.getEmpleados().stream())
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

```
// Ejercicio 62
```

```
Map<String, Long> ej62 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d -> (long) d.getEmpleados().size()));
```

```
// Ejercicio 63
```

```
Map<String, Double> ej63 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
```

```

        d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).max().
orElse(0));

```

// Ejercicio 64

```

List<String> ej64 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto,
Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

```

// Ejercicio 65

```

Map<String, List<Empleado>> ej65 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 2)
    .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue));

```

// Ejercicio 66

```

List<String> ej66 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria))
        .entrySet().stream()
        .filter(e ->
e.getValue().stream().mapToDouble(ProductoTienda::getPrecio).avera
ge().orElse(0) > 50)
        .flatMap(e -> e.getValue().stream())
        .map(ProductoTienda::getNombre)
        .sorted()
        .collect(Collectors.toList());

```

// Ejercicio 67

```

List<String> ej67 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingDouble(p ->
p.getProductos().stream()
        .mapToDouble(pr -> pr.getPrecio() *
pr.getStock()).sum()))
    .entrySet().stream()
    .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

```

// Ejercicio 68

```
List<String> ej68 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.summingInt(ProductoTienda::getStock)))
    .entrySet().stream()
    .filter(e -> e.getValue() > 5)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 69

```
Map<String, List<String>> ej69 = alumnos.stream()
    .filter(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0) >= 5)
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.mapping(AlumnoCurso::getNombre,
Collectors.toList())));
```

// Ejercicio 70

```
List<String> ej70 = departamentos.stream()
    .filter(d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).averag
e().orElse(0) > 1800)
    .flatMap(d -> {
        double media =
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).averag
e().orElse(0);
        return d.getEmpleados().stream()
            .filter(e -> e.getSalario() > media &&
e.getAntiguedad() >= 3);
    })
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

// Ejercicio 71

```
Map<String, List<String>> ej71 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.flatMapping(p ->
p.getProductos().stream().map(ProductoTienda::getCategoria).distan
ct(),
        Collectors.toList())));
```

// Ejercicio 72

```
Map<String, String> ej72 = productos.stream()
```

```

.collect(Collectors.groupingBy(ProductoTienda::getCategoria,
    Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPre
cio)),
    o ->
o.map(ProductoTienda::getNombre).orElse(""))));

```

// Ejercicio 73

```

List<String> ej73 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.averagingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0)))
    .entrySet().stream()
    .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

```

// Ejercicio 74

```

Map<String, List<String>> ej74 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().map(Empleado::getNombre).sorted().collec
t(Collectors.toList())));

```

// Ejercicio 75

```

String ej75 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingInt(p ->
p.getProductos().stream().mapToInt(ProductoTienda::getStock).sum()
)))
    .entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey).orElse("");
package REPASO7;

```

// Ejercicio 1

```

public class Caja<T> {
    private T contenido;
    public Caja(T contenido) { this.contenido = contenido; }
    public T getContenido() { return contenido; }
    public void setContenido(T contenido) { this.contenido =
contenido; }

```

```
    public void mostrarContenido() { System.out.println(contenido);  
}  
}
```

// Ejercicio 2

```
public class Par<K, V> {  
    private K clave;  
    private V valor;  
    public Par(K clave, V valor) { this.clave = clave; this.valor =  
valor; }  
    public K getClave() { return clave; }  
    public V getValor() { return valor; }  
    public String toString() { return clave + " -> " + valor; }  
}
```

// Ejercicio 3

```
public static <T> void mostrarArray(T[] array) {  
    for (T e : array) System.out.println(e);  
}
```

// Ejercicio 4

```
public static <T> T obtenerPrimero(List<T> lista) {  
    if (lista.isEmpty()) return null;  
    return lista.get(0);  
}
```

// Ejercicio 5

```
public static <T> int contarCoincidencias(List<T> lista, T  
elemento) {  
    int count = 0;  
    for (T e : lista) if (e.equals(elemento)) count++;  
    return count;  
}
```

// Ejercicio 6

```
public static <T extends Comparable<T>> T mayor(T a, T b) {  
    return a.compareTo(b) >= 0 ? a : b;  
}
```

// Ejercicio 7

```
class Repositorio<T> {  
    private List<T> lista = new ArrayList<>();  
    public void añadir(T e) { lista.add(e); }  
    public void eliminar(T e) { lista.remove(e); }  
    public List<T> obtenerTodos() { return lista; }  
    public T obtenerPorPosicion(int i) { return lista.get(i); }  
    public int contar() { return lista.size(); }  
}
```



```

// Ejercicio 8
public static <T> List<T> filtrar(List<T> lista, Predicate<T>
condicion) {
    return
lista.stream().filter(condicion).collect(Collectors.toList());
}

// Ejercicio 9
public static <T, R> List<R> transformar(List<T> lista,
Function<T, R> funcion) {
    return
lista.stream().map(funcion).collect(Collectors.toList());
}

// Ejercicio 10
public static <T> List<T> ordenarCopia(List<T> lista,
Comparator<T> comparador) {
    return
lista.stream().sorted(comparador).collect(Collectors.toList());
}

// PARTE 2 - REFERENCIAS DE MÉTODO

// Ejercicio 11
Function<String, String> f11 = String::toUpperCase;

// Ejercicio 12
Function<String, String> f12 = String::trim;

// Ejercicio 13
Consumer<String> c13 = System.out::println;

// Ejercicio 14
Function<Alumno, String> f14 = Alumno::getNombre;

// Ejercicio 15
Predicate<Alumno> p15 = Alumno::estaAprobado;

// Ejercicio 16
Consumer<Alumno> c16 = Alumno::mostrarFicha;

// Ejercicio 17
Function<String, String> f17 = UtilidadesTexto::limpiar;

// Ejercicio 18
Predicate<String> p18 = UtilidadesTexto::esLargo;

```

```

// Ejercicio 19
Impresora impresora = new Impresora();
Consumer<String> c19 = impresora::imprimir;

// Ejercicio 20
Supplier<ArrayList<String>> s20 = ArrayList::new;

// Ejercicio 21
// No se puede convertir directamente porque el constructor recibe 3
// parámetros
// y la lambda solo recibe 1, así que no hay referencia de método
// válida.
Function<String, Alumno> f21 = nombre -> new Alumno(nombre, 18,
5.0);

// Ejercicio 22
Comparator<Alumno> c22 = Comparator.comparing(Alumno::getNombre);

// Ejercicio 23
BiPredicate<String, String> p23 = String::equalsIgnoreCase;

// Ejercicio 24
Function<String, Integer> f24 = Integer::parseInt;

// Ejercicio 25
BiFunction<String, String, Integer> f25 =
String::compareToIgnoreCase;

// PARTE 3 - STREAMS

// Ejercicio 26
List<String> ej26 = productos.stream()
    .filter(p -> p.getPrecio() > 50)
    .map(ProductoTienda::getNombre)
    .sorted()
    .collect(Collectors.toList());

// Ejercicio 27
List<String> ej27 = productos.stream()
    .filter(p -> p.getCategoria().equals("Electrónica") &&
p.getStock() > 5)
    .map(ProductoTienda::getNombre)
    .collect(Collectors.toList());

// Ejercicio 28
double ej28 = productos.stream()
    .mapToDouble(ProductoTienda::getPrecio)
    .average().orElse(0);

```

// Ejercicio 29

```
Map<String, List<ProductoTienda>> ej29 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria));
```

// Ejercicio 30

```
Map<String, Double> ej30 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria,

Collectors.averagingDouble(ProductoTienda::getPrecio)));
```

// Ejercicio 31

```
Map<String, Integer> ej31 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria,

Collectors.summingInt(ProductoTienda::getStock)));
```

// Ejercicio 32

```
List<String> ej32 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 3)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 33

```
List<String> ej33 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 3)
    .flatMap(e -> e.getValue().stream())
    .map(ProductoTienda::getNombre)
    .sorted()
    .collect(Collectors.toList());
```

// Ejercicio 34

```
Map<String, Optional<ProductoTienda>> ej34 = productos.stream()

.collect(Collectors.groupingBy(ProductoTienda::getCategoria,

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPre
cio))));
```

// Ejercicio 35

```

Map<String, String> ej35 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.collectingAndThen(

Collectors.minBy(Comparator.comparingDouble(ProductoTienda::getPre
cio)),
        o ->
o.map(ProductoTienda::getNombre).orElse(""))));

// Ejercicio 36
List<ProductoTienda> ej36 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .collect(Collectors.toList());

// Ejercicio 37
List<String> ej37 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .map(ProductoTienda::getNombre)
    .distinct().sorted()
    .collect(Collectors.toList());

// Ejercicio 38
double ej38 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .mapToDouble(p -> p.getPrecio() * p.getStock())
    .sum();

// Ejercicio 39
Map<String, Double> ej39 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingDouble(p ->
p.getProductos().stream()
        .mapToDouble(pr -> pr.getPrecio() *
pr.getStock()).sum())));

// Ejercicio 40
List<String> ej40 = ej39.entrySet().stream()
    .filter(e -> e.getValue() > 500)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 41
Map<String, Long> ej41 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingLong(p ->
p.getProductos().size())));

```

// Ejercicio 42

```
List<String> ej42 = pedidos.stream()
    .filter(p -> p.getProductos().stream()
        .anyMatch(pr ->
            pr.getCategoria().equals("Electrónica")))
    .map(Pedido::getCliente)
    .distinct()
    .collect(Collectors.toList());
```

// Ejercicio 43

```
List<String> ej43 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente))
    .entrySet().stream()
    .filter(e -> e.getValue().size() > 1)
    .flatMap(e -> e.getValue().stream())
    .flatMap(p -> p.getProductos().stream())
    .map(ProductoTienda::getNombre)
    .distinct().sorted()
    .collect(Collectors.toList());
```

// Ejercicio 44

```
List<ProductoTienda> ej44 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())
    .filter(p -> p.getStock() > 1)

    .sorted(Comparator.comparingDouble(ProductoTienda::getPrecio).reversed())
    .collect(Collectors.toList());
```

// Ejercicio 45

```
Map<String, Optional<ProductoTienda>> ej45 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.flatMapping(p ->
            p.getProductos().stream(),
            Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPrecio)))));
```

// Ejercicio 46

```
List<Double> ej46 = alumnos.stream()
    .flatMap(a -> a.getNotas().stream())
    .collect(Collectors.toList());
```

// Ejercicio 47

```
double ej47 = alumnos.stream()
    .flatMapToDouble(a ->
        a.getNotas().stream().mapToDouble(Double::doubleValue))
    .average().orElse(0);
```

```

// Ejercicio 48
Map<String, Double> ej48 = alumnos.stream()
    .collect(Collectors.toMap(AlumnoCurso::getNombre,
        a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0)));

// Ejercicio 49
Map<String, Double> ej49 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.averagingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0))));

// Ejercicio 50
List<String> ej50 = alumnos.stream()
    .filter(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0) >= 7)
    .map(AlumnoCurso::getNombre)
    .collect(Collectors.toList());

// Ejercicio 51
String ej51 = alumnos.stream()
    .max(Comparator.comparingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0)))
    .map(AlumnoCurso::getNombre).orElse("");

// Ejercicio 52
Map<String, List<String>> ej52 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.mapping(AlumnoCurso::getNombre,
Collectors.toList())));

// Ejercicio 53
List<String> ej53 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso))
    .entrySet().stream()
    .filter(e -> e.getValue().stream().allMatch(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0) >= 5))
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

```

// Ejercicio 54

```
List<String> ej54 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso))
    .entrySet().stream()
    .filter(e -> e.getValue().stream().anyMatch(a ->
        a.getNotas().stream().anyMatch(n -> n < 5)))
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

// Ejercicio 55

```
List<Empleado> ej55 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.toList());
```

// Ejercicio 56

```
List<String> ej56 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .filter(e -> e.getSalario() > 2000)

    .sorted(Comparator.comparingDouble(Empleado::getSalario).reversed())
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

// Ejercicio 57

```
Map<String, List<Empleado>> ej57 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto));
```

// Ejercicio 58

```
Map<String, Double> ej58 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
    .orElse(0)));
```

// Ejercicio 59

```
String ej59 = ej58.entrySet().stream()
    .max(Map.Entry.comparingByValue())
    .map(Map.Entry::getKey).orElse("");
```

// Ejercicio 60

```
List<Empleado> ej60 = departamentos.stream()
    .filter(d -> d.getEmpleados().size() >= 3)
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.toList());
```

```
// Ejercicio 61
List<String> ej61 = departamentos.stream()
    .filter(d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).average()
    .orElse(0) > 2000)
    .flatMap(d -> d.getEmpleados().stream())
    .map(Empleado::getNombre)
    .collect(Collectors.toList());
```

```
// Ejercicio 62
Map<String, Long> ej62 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d -> (long) d.getEmpleados().size()));
```

```
// Ejercicio 63
Map<String, Double> ej63 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).max().
    orElse(0)));
```

```
// Ejercicio 64
List<String> ej64 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto,
Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
```

```
// Ejercicio 65
Map<String, List<Empleado>> ej65 = departamentos.stream()
    .flatMap(d -> d.getEmpleados().stream())
    .collect(Collectors.groupingBy(Empleado::getPuesto))
    .entrySet().stream()
    .filter(e -> e.getValue().size() >= 2)
    .collect(Collectors.toMap(Map.Entry::getKey,
Map.Entry::getValue));
```

```
// Ejercicio 66
List<String> ej66 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria))
        .entrySet().stream()
            .filter(e ->
e.getValue().stream().mapToDouble(ProductoTienda::getPrecio).average()
    .orElse(0) > 50)
```



```

        .flatMap(e -> e.getValue().stream())
        .map(ProductoTienda::getNombre)
        .sorted()
        .collect(Collectors.toList());

// Ejercicio 67
List<String> ej67 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.summingDouble(p ->
p.getProductos().stream()
    .mapToDouble(pr -> pr.getPrecio() *
pr.getStock()).sum()))))
    .entrySet().stream()
    .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 68
List<String> ej68 = pedidos.stream()
    .flatMap(p -> p.getProductos().stream())

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.summingInt(ProductoTienda::getStock)))
    .entrySet().stream()
    .filter(e -> e.getValue() > 5)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 69
Map<String, List<String>> ej69 = alumnos.stream()
    .filter(a ->
a.getNotas().stream().mapToDouble(Double::doubleValue).average().o
rElse(0) >= 5)
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.mapping(AlumnoCurso::getNombre,
Collectors.toList())));

// Ejercicio 70
List<String> ej70 = departamentos.stream()
    .filter(d ->
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).averag
e().orElse(0) > 1800)
    .flatMap(d -> {
        double media =
d.getEmpleados().stream().mapToDouble(Empleado::getSalario).averag
e().orElse(0);
        return d.getEmpleados().stream()

```

```

        .filter(e -> e.getSalario() > media &&
e.getAntiguedad() >= 3);
    })
    .map(Empleado::getNombre)
    .collect(Collectors.toList());

// Ejercicio 71
Map<String, List<String>> ej71 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,
        Collectors.flatMapping(p ->
p.getProductos().stream().map(ProductoTienda::getCategoria).distinct(),
        Collectors.toList())));

// Ejercicio 72
Map<String, String> ej72 = productos.stream()

    .collect(Collectors.groupingBy(ProductoTienda::getCategoria,
        Collectors.collectingAndThen(

Collectors.maxBy(Comparator.comparingDouble(ProductoTienda::getPrecio)),
        o ->
o.map(ProductoTienda::getNombre).orElse(""))));

// Ejercicio 73
List<String> ej73 = alumnos.stream()
    .collect(Collectors.groupingBy(AlumnoCurso::getCurso,
        Collectors.averagingDouble(a ->

a.getNotas().stream().mapToDouble(Double::doubleValue).average().orElse(0))))
    .entrySet().stream()
    .sorted(Map.Entry.<String,
Double>comparingByValue().reversed())
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());

// Ejercicio 74
Map<String, List<String>> ej74 = departamentos.stream()
    .collect(Collectors.toMap(Departamento::getNombre,
        d ->
d.getEmpleados().stream().map(Empleado::getNombre).sorted().collect(Collectors.toList())));

// Ejercicio 75
String ej75 = pedidos.stream()
    .collect(Collectors.groupingBy(Pedido::getCliente,

```

```
                Collectors.summingInt(p ->
p.getProductos().stream().mapToInt(ProductoTienda::getStock).sum()
)))
        .entrySet().stream()
        .max(Map.Entry.comparingByValue())
        .map(Map.Entry::getKey).orElse("");
```